Instructions : `sample_mcmc()`

Joakim De Rambures

December 2025

Overview of `sample_mcmc()`: The function `sample_mcmc()` runs an MCMC sampler for a given posterior model from the Posterior Database. It wraps several sampling algorithms (random-walk, Langevin, Barker's method, etc.) under a common interface. The user specifies which algorithm ("method") to use, and `sample_mcmc()` will automatically initialize the model, run the chosen sampler for the desired number of warm-up (adaptation) and main iterations, and return both the sampled chains and diagnostic metrics. It is a convenience function for comparing different MCMC algorithms on the same target distribution.

Supported Sampling Methods: The `method` argument selects one of five MCMC algorithms implemented:

- `"amh"` – *Adaptive Metropolis-Hastings*: A Gaussian random-walk Metropolis-Hastings sampler with adaptation. During warm-up, it adjusts the proposal scale (step size) to target a given acceptance rate (e.g. via Robbins-Monro stochastic approximation). This is a classic adaptive random walk MH that tunes the covariance of the proposal based on past samples.
- `"amala"` – *Adaptive MALA (Metropolis-Adjusted Langevin Algorithm)*: A gradient-informed Metropolis sampler. It uses Langevin dynamics (gradient of the log-density) to propose moves, combining the efficiency of MALA with adaptation of the step size. During warm-up, the algorithm adapts its scale and possibly shape to achieve a desired acceptance probability.
- `"barker"` – *Barker's algorithm (Locally Balanced Proposal)*: A robust gradient-based MCMC method inspired by the Barker accept-reject rule. It uses the gradient of the log-density but with a smoother acceptance probability (from Barker, 1965, Livingstone Zanella, 2022) instead of the usual Metropolis rule. This can improve robustness to step size tuning. The Barker proposal tends to be more forgiving in high dimensions or poorly scaled problems.
- `"bimodal"` – *Bimodal Barker proposal*: A variant of the Barker sampler that uses a bimodal proposal distribution. In practice this means the proposal is a mixture of two Gaussian steps (controlled by a parameter `sigma`) to better explore multi-modal target distributions. It's useful for targets with separate modes, helping the chain jump between modes more easily than a single Gaussian proposal would.
- `"hmc"` – *Hamiltonian Monte Carlo*: A dynamic Hamiltonian Monte Carlo sampler (using the `rmcmc` package's `hamiltonian_proposal`). It simulates Hamiltonian dynamics with a leapfrog integrator to propose distant moves in parameter space. The user can run fixed-length trajectories or randomized trajectory lengths, and optionally employ partial momentum refreshment. This method can greatly improve efficiency for well-behaved posteriors by reducing random walk behavior.

All samplers share the use of a common target log-density (and gradient when needed) obtained from the specified posterior model. The function automatically constructs the target distribution from the Stan model code and data in the PosteriorDB.

Key Arguments and Options: Below are important arguments of `sample_mcmc()` and their roles:

- `name`: The name of the posterior to sample (must correspond to a model in the Posterior Database). This provides the model code, data, and potentially reference ("ground truth") posterior draws for diagnostics.
- `warm_up` and `main`: Number of iterations for the warm-up (adaptation) phase and the main sampling phase, respectively. During warm-up the sampler's adaptive algorithms adjust tuning parameters, samples from this phase are by default discarded from the returned chains.
- `chains`: The number of parallel MCMC chains to run. Each chain will be initialized at a different starting point (the code jitters the initial state slightly for each chain) and run independently.
- `thin`: Thinning interval for the output samples (applied after warm-up). For example, `thin=2` would keep every 2nd draw of the main iterations.

- `scale`: The base step size or scale parameter for the proposal. Each sampler interprets this scale in context (e.g. step size for HMC, standard deviation of random-walk proposals, etc.). If left `NULL`, a default scale is chosen based on dimension and sampler type (for instance, Adaptive MH uses $2.38/\sqrt{d}$ by default as per optimal scaling theory).
- `adapters`: Controls whether adaptive tuning is applied during warm-up, and how. This can be specified in two ways:
  - As a preset name `"stoch_approx_adapt"` or `"dual_averaging_adapt"`. These strings activate built-in adapter lists. **Stochastic approximation adaptation** uses a Robbins-Monro scheme that adjusts the proposal scale every iteration to approach a target acceptance rate (often 0.44 for random-walk, 0.574 for Barker, etc.), combined with a *variance adapter* that scales each parameter's step size according to the empirical variance of the chain (to better handle different scales in different dimensions). **Dual averaging adaptation** uses Stan's style of adaptation (Nesterov's dual averaging) for the step size, again targeting a given acceptance probability, plus the same variance-based shape adaptation. Both adaptation schemes have hyperparameters: κ (learning rate decay), γ and `iteration_offset` (for dual averaging stability), and `tau_star` (target acceptance probability). These are exposed as arguments with default values recommended by the literature (e.g. $\kappa = 0.6$, $\gamma = 0.05$).
  - As a custom list of adapter objects. For advanced users, you can supply any list of adapters compatible with the `rmcmc::sample_chain` function. Each adapter in the list should have methods to update the proposal's parameters during warm-up. Typical adapters include those that adjust scale or covariance based on acceptance rate or on tracked variance. If `adapters=NULL`, no adaptation is done (the sampler runs with fixed `scale`/`shape` throughout).
- `HMC-specific options (n_step, sample_n_step, sample_auxiliary)`: These arguments fine-tune HMC behavior. By default, if `method="hmc"` and no trajectory options are given, the number of leapfrog steps will be randomized each iteration (to avoid periodic behavior), usually drawn uniformly from a range (e.g. 5 to 100). The user can override this by specifying:
  - `n_step`: Fix a constant number of leapfrog steps per HMC trajectory. For example, `n_step=50` means every iteration the integrator does 50 steps. This turns off random trajectory length sampling.
  - `sample_n_step`: If set to TRUE, the trajectory length is randomly chosen each iteration (the default behavior). You may also provide a custom function to draw the number of steps if desired. Setting this to FALSE (or NULL) while leaving `n_step` unspecified will default to a fixed single-step trajectory (which essentially reduces HMC to a Langevin proposal).
  - `sample_auxiliary`: Controls momentum refreshment. By default, HMC fully resamples a new momentum at the start of each trajectory. If `sample_auxiliary=TRUE`, the sampler will only partially refresh the momentum between iterations (using a "partial momentum update" where the new momentum is a mix of the old momentum and fresh noise). This technique can improve sampling efficiency in some cases by preserving some momentum between trajectories (it introduces a second-order Markov chain). If `sample_auxiliary=FALSE` (or NULL), momenta are fully refreshed (standard HMC behavior). This option is ignored for other methods.
- `sigma`: Used only for the `"bimodal"` Barker sampler. It controls the separation of the two modes in the bimodal proposal distribution. A smaller σ (e.g. 0.1) makes the two proposal components closer together, a larger σ spreads them further apart. Tuning this may help the chain jump between widely separated modes in the target distribution.
- `seed`: Random seed for reproducibility. This is passed to the Stan model and also used for R's random number generation to ensure identical results when re-running the function.
- `plot` and `save_plot`: If `plot=TRUE`, the function will produce basic trace plots of the running diagnostic curves (see below) in the active graphics device. If `save_plot=TRUE`, it will also save PDFs of these plots (named by the model, e.g. `<name>_runningMSE.pdf`, etc.) in the working directory. In typical non-interactive usage (or when running many chains programmatically), these can be turned off to save time.

Most of these arguments have reasonable defaults. For instance, by default two chains are run with `warm_up=100` and `main=1000` iterations, and if `scale` is not provided it is set to a standard recommendation based on dimension (e.g. 0.8 or $2.38/\sqrt{d}$ depending on method). A user can often simply choose a method and a posterior name and rely on defaults for a quick run.

Returned Value and Diagnostics: `sample_mcmc()` returns a list with three main components:

- `$mcl`: An `mcmc.list` containing the posterior draws from all chains (after warm-up and thinning). This is a coda `mcmc.list` object, so it can be directly used with `coda` or `bayesplot` functions for further analysis or plotting. Each chain's samples of all model parameters are included.
- `$diag_basic`: A data frame of basic diagnostics for each parameter. This is produced by `posterior::summarise_draws` under the hood. For each model parameter, it reports summary statistics and convergence diagnostics such as:
  - Posterior mean, standard deviation, and median (and possibly MAD).
  - Quantiles (e.g. 5%, 50%, 95% quantiles for uncertainty intervals).
  - Monte Carlo standard error of the mean (`MCSE_mean`) : an estimate of the error due to finite sampling.
  - Effective sample size for bulk and tail (`ess_bulk`, `ess_tail`) : indicating how many independent samples the chain is roughly equivalent to, for central tendency and tail estimation respectively.
  - Gelman-Rubin $\hat{R}$ for convergence (values near 1.0 indicate good mixing between chains).

  Additionally, `diag_basic` is augmented with a few overall metrics: `ess_min_run` (the minimum ESS among parameters, which identifies the bottleneck parameter for mixing), `runtime_sec` (the total wall-clock time the sampling took), and `ess_min_per_sec` (the minimum ESS divided by runtime, an efficiency metric indicating how many effective samples per second were obtained for the slowest-mixing parameter). These give a sense of sampling efficiency and performance.
- `$diag_extra`: A list of advanced diagnostics, returned only if reference posterior values are available for the model (i.e. if the PosteriorDB provides "ground truth" posterior means and variances from a high-accuracy method such as long NUTS runs). When reference data exists, the function computes:
  - **MSE curves** (*running mean squared error of the posterior mean estimate*): At each iteration t, it computes the MSE between the chain's current mean estimate for each parameter and the true posterior mean, averaged across parameters. This shows how quickly the chain's estimates approach the true values over time. The result is a vector of length equal to the number of main iterations, which can be plotted to diagnose convergence (decreasing MSE indicates the chain's mean is approaching the truth).
  - **DT curves** (*running log-variance distance*): Similarly, at each iteration it computes the distance between the log of the chain's current variance estimate for each parameter and the log of the true posterior variances, then takes the root mean square across parameters. This "DT" (distance in log-variance) curve starts high if the chain's variance estimates are off and should decrease as the chain converges and captures the correct spread of the posterior. This is another convergence diagnostic, focused on second moments.
  - **ESJD** (*expected squared jumping distance*) statistics: This measures how far the chain moves, on average, in parameter space per iteration. For each chain, the ESJD is computed as the mean of the squared Euclidean distance between consecutive states. A higher ESJD generally indicates better mixing (the chain explores farther rather than getting stuck). The function provides per-chain ESJD values and also a running average of ESJD over iterations. This helps in comparing how different samplers or tuning settings traverse the space. For example, HMC might have a larger ESJD than a random walk MH for the same acceptance rate, reflecting its ability to make bigger jumps.

  These advanced diagnostics are contained in `diag_extra` as numeric vectors or small matrices. For instance, `diag_extra$curves` holds a list with elements `$mse`, `$dt`, and `$esjd` which are the running curves (averaged across chains) for those metrics. If reference draws are not available for the chosen model, `diag_extra` may be `NULL` or its entries `NULL`, and a message is printed indicating that advanced diagnostics could not be computed.

Using the returned diagnostics, one can assess both convergence and efficiency of the sampler. The basic table allows checking for $\hat{R} < 1.1$ and sufficient ESS, while the extra diagnostics (when available) show how quickly the chain converges to the correct distribution and how efficiently it explores the space.

Limitations and Requirements: The function is tailored to the PosteriorDB setup, so it requires that the `name` corresponds to a valid posterior in the database and that the model can be instantiated via Stan (the code uses `bridgestan` under the hood to create a `StanModel` and evaluate log densities). This means you must have the PosteriorDB installed locally and accessible (via `pdb_local()` initialization as shown in the code) and also have a working Stan toolchain (since gradients and log-densities are needed for the gradient-based samplers).

Another limitation is that the most insightful diagnostics (MSE, DT, etc.) are only available if the reference posterior is known. Many models in PosteriorDB have reference draws (often drawn from long NUTS runs or analytic solutions), but if a model lacks this, `sample_mcmc` will still run the chains but simply won't report MSE/DT curves. Moreover, `sample_mcmc` is not a general replacement for a full MCMC toolkit: it is designed for research and comparison of specific algorithms, so the interface is somewhat fixed (for example, the set of samplers is hard-coded, and the target must be a Stan model from PosteriorDB). It does not implement the No-U-Turn Sampler (NUTS) in this function (though a separate helper `run_NUTS()` is present in the code for reference). Finally, one should note that while the function automates many things, it does minimal error-checking on inputs, the user should ensure the `name` and parameters make sense (e.g., using HMC-specific options only when `method="hmc"`).

Example Usages: Here we illustrate a few example calls to `sample_mcmc()` to demonstrate how it can be used in practice. These examples assume that the PosteriorDB is set up and that the model "`eight_schools-eight_schools_nonce` (a classic hierarchical modeling example) is available in it.

```
Example 1: Basic adaptive Metropolis-Hastings on a given model

result1 <- sample_mcmc(method = "amh",
name = "eight_schools-eight_schools_noncentered",
warm_up = 500,
main = 2000,
chains = 2,
adapters = "stoch_approx_adapt")
```

In this call, we run an adaptive random-walk MH on the eight schools model, with 500 warm-up iterations and 2000 main iterations for 2 chains. We chose the stochastic approximation adapter to automatically tune the proposal scale. The returned `result1` will contain the two chains of samples and diagnostics. One can inspect `result1$diag_basic` to see the $\hat{R}$ and ESS for all school parameters and hyperparameters, and use `coda::traceplot(result1$mcl)` or `bayesplot` to visualize convergence.

```
Example 2: Hamiltonian Monte Carlo with fixed trajectory length

result2 <- sample_mcmc(method = "hmc",
name = "eight_schools-eight_schools_noncentered",
warm_up = 1000,
main = 5000,
chains = 2,
adapters = "dual_averaging_adapt",
n_step = 10, # fixed 10 leapfrog steps
sample_n_step = FALSE, # no random step count
sample_auxiliary = FALSE) # full momentum refresh
```

Here we run HMC on the same model. We use 1000 warm-up and 5000 sampling iterations. We enable dual averaging adaptation to tune the step size during warm-up (targeting a typical ≈ 0.8 acceptance for HMC). We explicitly set `n_step=10` and turn off `sample_n_step`, which means each HMC trajectory will consist of 10 leapfrog steps (a fixed trajectory length). We also set `sample_auxiliary=FALSE` so that momenta are fully resampled each iteration (standard HMC). The result `result2` will likely show very high effective sample sizes and $\hat{R} \approx 1.0$ for all parameters if HMC is performing well. One can check `result2$diag_extra` as well, since eight schools has reference draws, this list will contain MSE and ESJD curves. Plotting the `result2$diag_extra$curves$mse` vector, for example, would show how quickly the running mean estimates approach the true posterior means.

```
Example 3: Barker's algorithm with partial momentum refresh (auxiliary) HMC

result3 <- sample_mcmc(method = "hmc",
name = "eight_schools-eight_schools_noncentered",
warm_up = 500,
main = 2000,
chains = 2,
```

```
adapters = "stoch_approx_adapt",
sample_n_step = TRUE, # randomize trajectory length
sample_auxiliary = TRUE) # use partial momentum refresh
```

This example actually combines features: we still use `method="hmc"`, but we illustrate a scenario akin to a generalized HMC. Here we allow the number of leapfrog steps to be randomly chosen each iteration (`sample_n_step=TRUE`, with the default range of 5–100 steps), and we set `sample_auxiliary=TRUE` to enable partial momentum updates. We also use the stochastic approximation adapter for step size and variance adaptation. This configuration is more experimental, it may yield better mixing on some targets by adding randomness to trajectory lengths and not completely forgetting the momentum each time. The returned diagnostics can be compared to those of standard HMC. For instance, one could compare the ESJD from `result3$diag_extra` to that from `result2` to see if the partial momentum refresh increased the jumping distance.

These examples demonstrate how `sample_mcmc()` can be flexibly configured. By adjusting the `method` and options, one can explore different MCMC algorithms on the same model and use the unified diagnostics output to compare their performance.